

n-grams smoothing

Aim: To write a python program to implement n-grams smoothing technique

Description: Incorporating smoothing into a bi-gram model helps to handle cases where some bi-grams may not appear in the training corpus. Simple Additive Smoothing (also known as Laplace Smoothing) method. This program will build n-grams from a given text and apply smoothing to handle zero probabilities.

Implementation of bi-gram model with Laplace smoothing:

Steps to Incorporate Smoothing

- Count the Bi-grams and Unigrams:
 - Count occurrences of each bi-gram.
 - Count occurrences of each unigram (individual word).
- Calculate Probabilities with Laplace Smoothing:
 - For each bi-gram, apply the formula:

$$P(w_2|w_1) = \frac{(count(w_1, w_2) + 1)}{(count(w_1) + V)}$$

- Here, V is the size of the vocabulary (total number of unique words).

Steps are

1. **get_ngrams:** Tokenizes the input text and generates n-grams.
2. **count_ngrams:** Counts the occurrences of each n-gram.
3. **laplace_smoothing:** Applies Laplace smoothing to the n-gram counts. It adds 1 to each count and normalizes by the total count plus the vocabulary size to avoid zero probabilities.
4. **build_vocabulary:** Builds a set of unique tokens (vocabulary) from the input text..

Program:

```
import collections
import nltk
from nltk.util import ngrams
from nltk.tokenize import word_tokenize

def get_ngrams(text, n):
    """Generate n-grams from the given text."""
    tokens = word_tokenize(text.lower())
    n_grams = ngrams(tokens, n)
    return list(n_grams)

def count_ngrams(ngrams):
    """Count the occurrences of each n-gram."""
    ngram_counts = collections.Counter(ngrams)
    return ngram_counts

def laplace_smoothing(ngram_counts, unigram_counts, vocab_size, n):
    """Apply Laplace smoothing to the n-gram counts."""
```

```

smoothed_probs = {}

for ngram in ngram_counts:
    context = ngram[:-1]
    context_count = unigram_counts[context] if n > 1 else sum(unigram_counts.values())
    smoothed_probs[ngram] = (ngram_counts[ngram] + 1) / (context_count + vocab_size)

return smoothed_probs

def build_vocabulary(text):
    """Build a vocabulary from the given text."""
    tokens = word_tokenize(text.lower())
    vocab = set(tokens)
    return vocab

# Example text
text = "this is a sample text with several words this is another sample text with some different words"

# Set n for n-grams
n = 2 # For example, bi-grams

# Generate n-grams
ngrams_list = get_ngrams(text, n)
unigrams_list = get_ngrams(text, 1)

# Count n-grams and unigrams
ngram_counts = count_ngrams(ngrams_list)
unigram_counts = count_ngrams(unigrams_list)

# Build vocabulary
vocab = build_vocabulary(text)
vocab_size = len(vocab)

# Apply Laplace smoothing
smoothed_probs = laplace_smoothing(ngram_counts, unigram_counts, vocab_size, n)

# Output results
print("N-grams with their smoothed probabilities:")
for ngram, prob in smoothed_probs.items():
    print(f'{ngram}: {prob:.6f}')

```

Output:

```

N-grams with their smoothed probabilities:
('this', 'is'): 0.230769
('is', 'a'): 0.153846
('a', 'sample'): 0.166667
('sample', 'text'): 0.230769
('text', 'with'): 0.230769
('with', 'several'): 0.153846
('several', 'words'): 0.166667

```

```
('words', 'this'): 0.153846  
('is', 'another'): 0.153846  
('another', 'sample'): 0.166667  
('with', 'some'): 0.153846  
('some', 'different'): 0.166667  
('different', 'words'): 0.166667
```